

NAOS-post-pass-report-generation

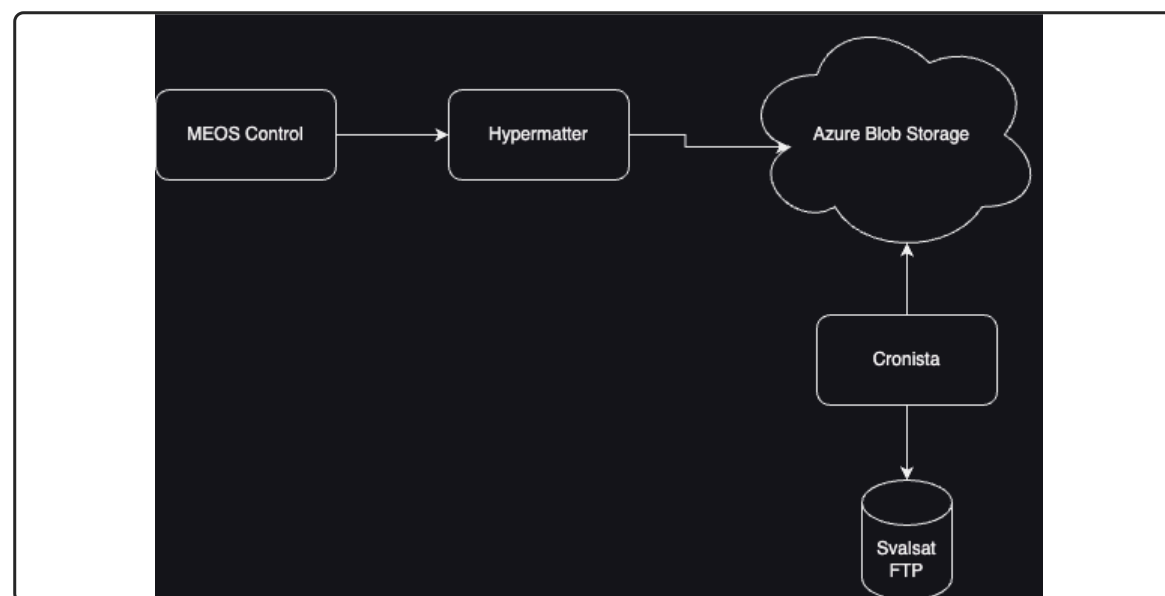
The NAOS project require post pass reports to be delivered on svalsat FTP server. This article entail the services used to produce these reports and shall provide some hints for where to look when issues are reported.

Post pass report content

The post pass report contain several sections:

- X-band files acquired on Cortex HDR
- Summary of the S-band telemetry data acquired

Architecture



NAOS-post-pass-report-g...

- Post pass report content
- Architecture
- Container description
- Meos Control
- Hypermatter-py
- Cronista

Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camriere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWX

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

Container description

This section provide a brief overview of what the containers listed in the #Architecture section do.

Meos Control

MEOS Control is the control system in this context, and is responsible for pass execution. When LOS is reached, it is also responsible for providing S-band telemetry data and doppler recordings to azure blob storage. It is enabled to do so by using HTTP POST requests against the hypermatter service.

Hypermatter-py

Hypermatter is a service that move data from A to B. For NAOS it also produce the S-band telemetry data extract, or `crtinfo` if you will. The `crtinfo` data file is written to azure blob storage named `stsitereports`.

When hypermatter receive the S-band recordings from MEOS Control, hypermatter produce the `crtinfo` extract and push that information to azure blob storage with the contact ID as file name.

For doppler recordings, the doppler recording is uploaded to `stsitereports` storage account, and the `doppler` container. Again, the contact ID is used as file name.

The hypermatter-py repository can be found here:

<https://github.com/kongsberg-satellite-services/maestro/tree/master/modules/hypermatter-py>

The hypermatter-py API is documented in the repository README

Cronista

Cronista read the doppler data, crtinfo and all the needed information from `stsitereports` and assemble the post-pass reports. The full report is written to svalsat FTP at a configurable file path.

The cronista user interface can be accessed here: <http://prefect.prefect.production.igns.ksat.site:3000/>

The cronista repository is here: <https://github.com/kongsberg-satellite-services/cronista>

The cronista configuration is here: https://github.com/kongsberg-satellite-services/cronista/blob/master/configs/prod/postpass_reports.yml